- Y. Liang Daniel, Introduction to Java Programming

Java

print line

System.out.println(" "); ekranı yazdırıyo.

Sonuna otomatik \n ekliyo gibi çalışır. Sadece print dersen alt satıra geçmez

String metin = JOptionPane.showInputDialog(null, "Ilk sayıyı gir:");

showMessageDialog(null, "Sonuç:");

pencere çıkarıyor karşına. Inputta dan Scanf gibi. Message ise sadece mesaj veriyor.

```
import java.util.scanner
import javax.swing.JOptionPane;

public class Sinifim {
      public static void main (String[] args){
            double r,a;
            Scanner ali = new Scanner(System.in);
            System.out.print("Dairenin yarıçapını girin: ");
            r = ali.nextDouble();
            a = Math.PI*Math.pow(r, 2);
            String metin = "Yarıçapı " + r + " olan dairenin alanı " + a +" dir.";
            JOptionPane.showMessageDialog(null, metin, "Sonuç
budur!",JOptionPane.ERROR_MESSAGE);
      }
}
```

→ importlayamıyosan ampule basıp sondan ikinciye tıklıyoruz.

pencerede yazacak şey

→ pencerenin başlığı

"bu diyalog penceresi bu pencere ile bağlantılı mı?"

logo

↓(Başka örnek:) INFORMATION_MESSAGE

kelime.ali.next(); // bir boşluk veya tab veya /n görene kadar okur.

double c = (double) 3/2; → 1,5 çıkarır.

double c = 1/2; → 0.0 çıkarır.

int int

byte b=(byte)i; //explicit casting

yazmak zorundasın

Stringleri Yazdırma

`System.out.println(str1.lengt());` string'in uzunluğunu yazar.

`System.out.println(str1.charAt(0));` ilk karakterini yazar

`System.out.println(str1.equals("hello"));` false döner

`System.out.println(str1.toUpperCase());` hepsini büyük yazar

`String altDizi = str1.substring(1, 4);` baş ↑ bitiş ↑ (dahil değil)

`substring(1);` ↓ başlayıp sonuna kadar

`System.out.println(altDizi);`

Kullanıcıdan Veri Alma

```
Scanne scanner = new Scanner(System.in);
System.out.println("Isminizi girin: ");
String isim = scanner.nextLine();
System.out.println(isim);
```

— `int sayi = Integer.parseInt(deger);`

String'i int'e ↑ çevirir.

Integer class'ının alt metodudur (statictir)

★ Boolean veriler C'de yok. Javada var.
Sonucu true yada false olabilir.

★ Javada ~~while(1)~~ diye bir şey yok while(true) var

★ `if((number %3 == 0) ^ (number %2 == 0))`

↓ yada (sayı 2 yada 3'e bölünürse true olacak, ikisi birdense false)

★ Javada Srand.time(null) yapmana gerek yok. Her çalıştığında farklı bir şey çıkarır.

```
int r= (int) (Math.random()*90)+10;
```
(10 ve 99 arası sayı belirler)

★ Javadaki switch-case'lerde "Default" yapmak zorunlu değil

★ %b → boolean değerler için

Metotlar (C'deki fonksiyonlar gibi)

★ Bir metot statikse sadece statik metotları çağırabilir.

★ Başka bir sınıftaki "static" olmayan bir metodu çağırabilmek için o başka sınıftan bir nesne yaratmamız, çağrıyı bu nesne üstünden yapmamız gerekir.

```
class Yardimci{
  void konsola_yaz(String e){
        System.out.println(e);
  }
}

public static void main(String[] args){
Yardimci.konsola yaz("yey" + " selam");
}
```

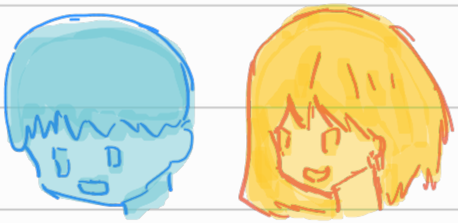
```
int[] dizi1 = new int[5];
```
Javada dizi tanımlama

```
int* a= (int*) malloc(8*sizeof(int));
int[] dizi4 = {7,5,-1,2,3};
for(int i=0; i<dizi6.length; i++)
     System.out.print(dizi6[i]);
```

Javada %lf yok
%f var

```
dizi.length;
```
// dizilerde sadece length, stringlerin harf sayısıysa length();

StringTokenizer tokenizer = new StringTokenizer(metin, " .,;!?");

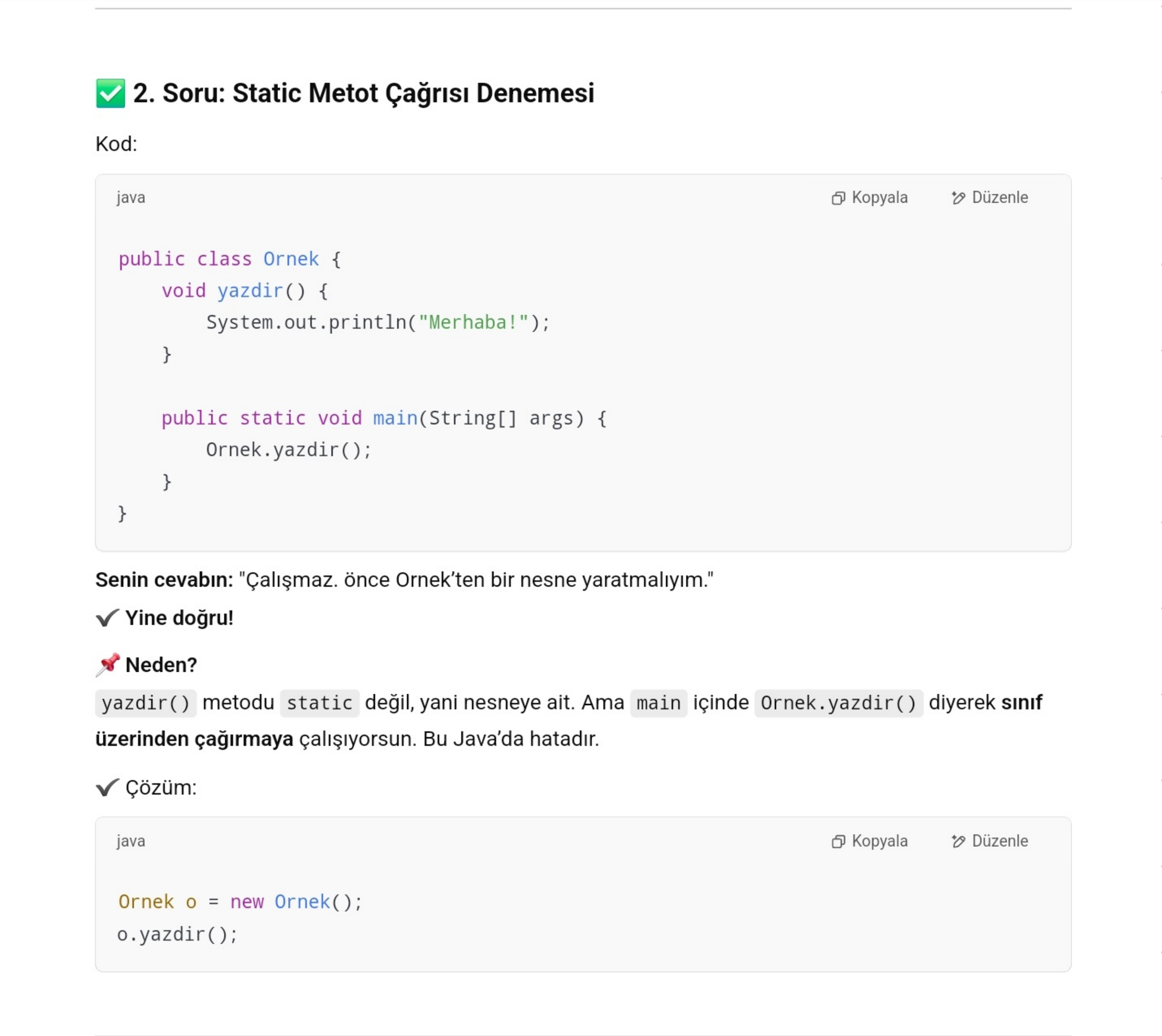


↓ metni kelimelerine ayırır

"Static" olunca bir sınıfa ait oluyor. Olmayınca ise bir nesneye ait oluyor.

- Sınıfa ait bir yerden (main) doğrudan nesneye ait değişkene ulaşılamaz.

↓ statictir



✅ 2. Soru: Static Metot Çağrısı Denemesi

Kod:

```java
public class Ornek {
    void yazdir() {
        System.out.println("Merhaba!");
    }

    public static void main(String[] args) {
        Ornek.yazdir();
    }
}
```

Senin cevabın: "Çalışmaz. önce Ornek'ten bir nesne yaratmalıyım."

✔ **Yine doğru!**

📌 **Neden?**

`yazdir()` metodu `static` değil, yani nesneye ait. Ama `main` içinde `Ornek.yazdir()` diyerek **sınıf üzerinden çağırmaya** çalışıyorsun. Bu Java'da hatadır.

✔ Çözüm:

```java
Ornek o = new Ornek();
o.yazdir();
```

Yani şey gibi düşün: Static bi hayal ürününe ait. main de static mesela. Ve hayaller, gerçek nesnelere erişemiyo.

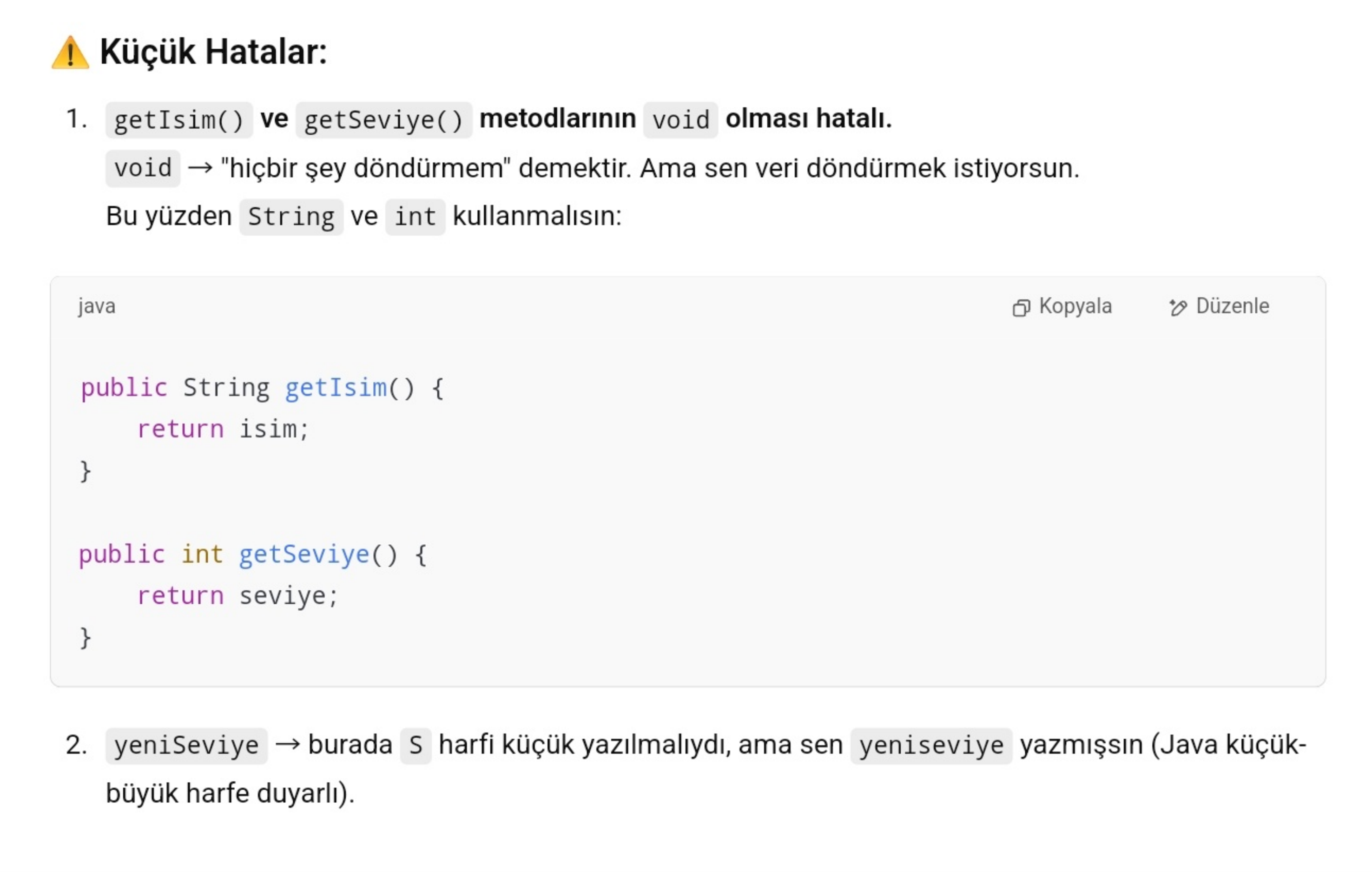
Static → class'a ait (hayal, plan)

Getter-Setter

Private değişkenler değiştirebilmek için kullanılıyo
Aynı zamanda bir metodturlar. İsimleri özelleşmiş sadece.

Kapsülleme (encapsulation)

Java'nın OOP felsefesinin temeli. "Nesnenin içini sakla, dışarıya sadece ne gerekiyorsa onu göster."
Bu mantık C'de çok zor sağlanıyordu. Java bu işi biraz sıkı tutuyor.

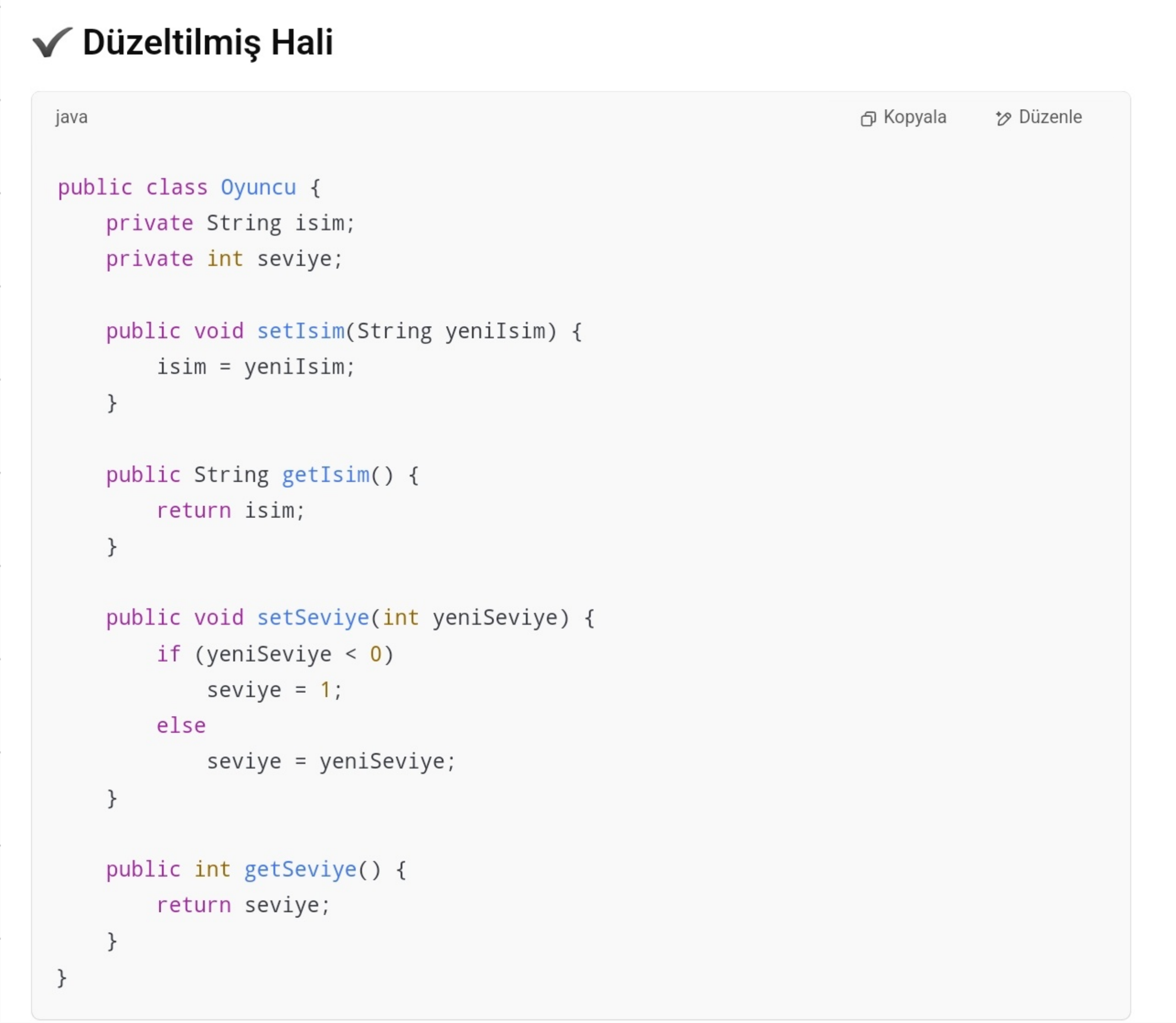


⚠️ Küçük Hatalar:

1. `getIsim()` **ve** `getSeviye()` **metodlarının** `void` **olması hatalı.**
   `void` → "hiçbir şey döndürmem" demektir. Ama sen veri döndürmek istiyorsun.
   Bu yüzden `String` ve `int` kullanmalısın:

```java
public String getIsim() {
    return isim;
}

public int getSeviye() {
    return seviye;
}
```

2. `yeniSeviye` → burada `S` harfi küçük yazılmalıydı, ama sen `yeniseviye` yazmışsın (Java küçük-büyük harfe duyarlı).

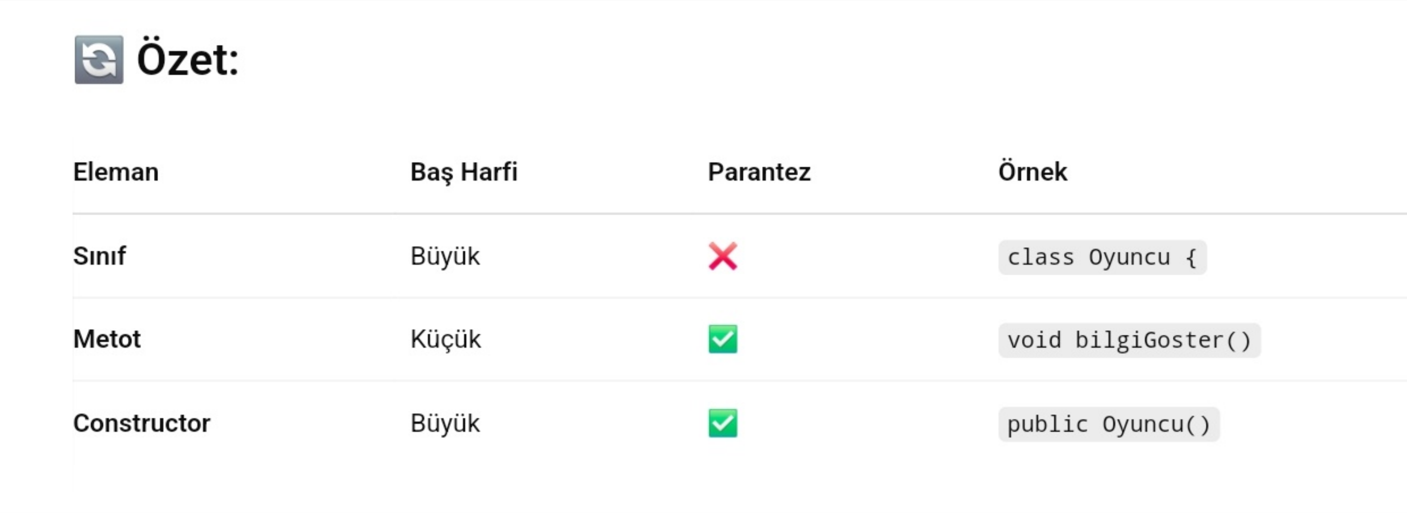


✔ Düzeltilmiş Hali

```java
public class Oyuncu {
    private String isim;
    private int seviye;

    public void setIsim(String yeniIsim) {
        isim = yeniIsim;
    }

    public String getIsim() {
        return isim;
    }

    public void setSeviye(int yeniSeviye) {
        if (yeniSeviye < 0)
            seviye = 1;
        else
            seviye = yeniSeviye;
    }

    public int getSeviye() {
        return seviye;
    }
}
```

```
public class Oyuncu {
   private String isim;
   private int seviye = 1;

   public Oyuncu(String isim) {
      this.isim = isim;
   }

   public void bilgiGoster() {
      System.out.println("Oyuncu: " + isim + " | Seviye: " + seviye);
   }

   public void setSeviye(int yeniSeviye) {
      if (yeniSeviye < 0)
         seviye = 1;
      else
         seviye = yeniSeviye;
   }
```

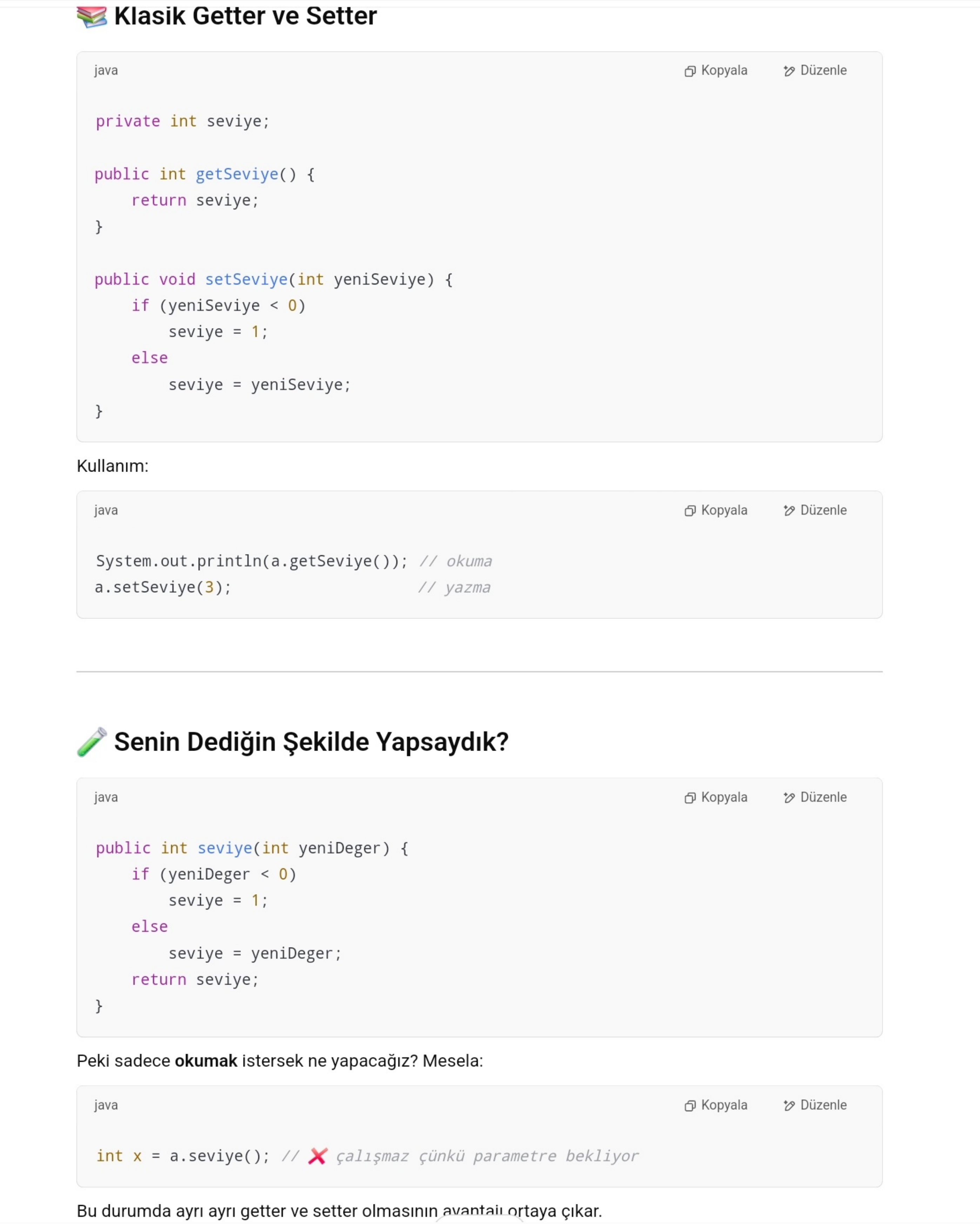
```
    public int getSeviye() {
        return seviye;
    }

    public void setIsim(String yeniIsim) {
        isim = yeniIsim;
    }

    public String getIsim() {
        return isim;
    }
}

public class Main {
    public static void main(String[] args) {
        Oyuncu a = new Oyuncu("Ayse");
        a.setSeviye(5);

        Oyuncu b = new Oyuncu("Kaan");
        b.setSeviye(-1); // bu -1 verildiği için seviye otomatik 1 olacak

        a.bilgiGoster();
        b.bilgiGoster();
```



🔄 Özet:

Eleman	Baş Harfi	Parantez	Örnek
Sınıf	Büyük	❌	`class Oyuncu {`
Metot	Küçük	✅	`void bilgiGoster()`
Constructor	Büyük	✅	`public Oyuncu()`

```
    }
}
```



📚 Klasik Getter ve Setter

```java
private int seviye;

public int getSeviye() {
    return seviye;
}

public void setSeviye(int yeniSeviye) {
    if (yeniSeviye < 0)
        seviye = 1;
    else
        seviye = yeniSeviye;
}
```

Kullanım:

```java
System.out.println(a.getSeviye()); // okuma
a.setSeviye(3);                    // yazma
```

🧪 Senin Dediğin Şekilde Yapsaydık?

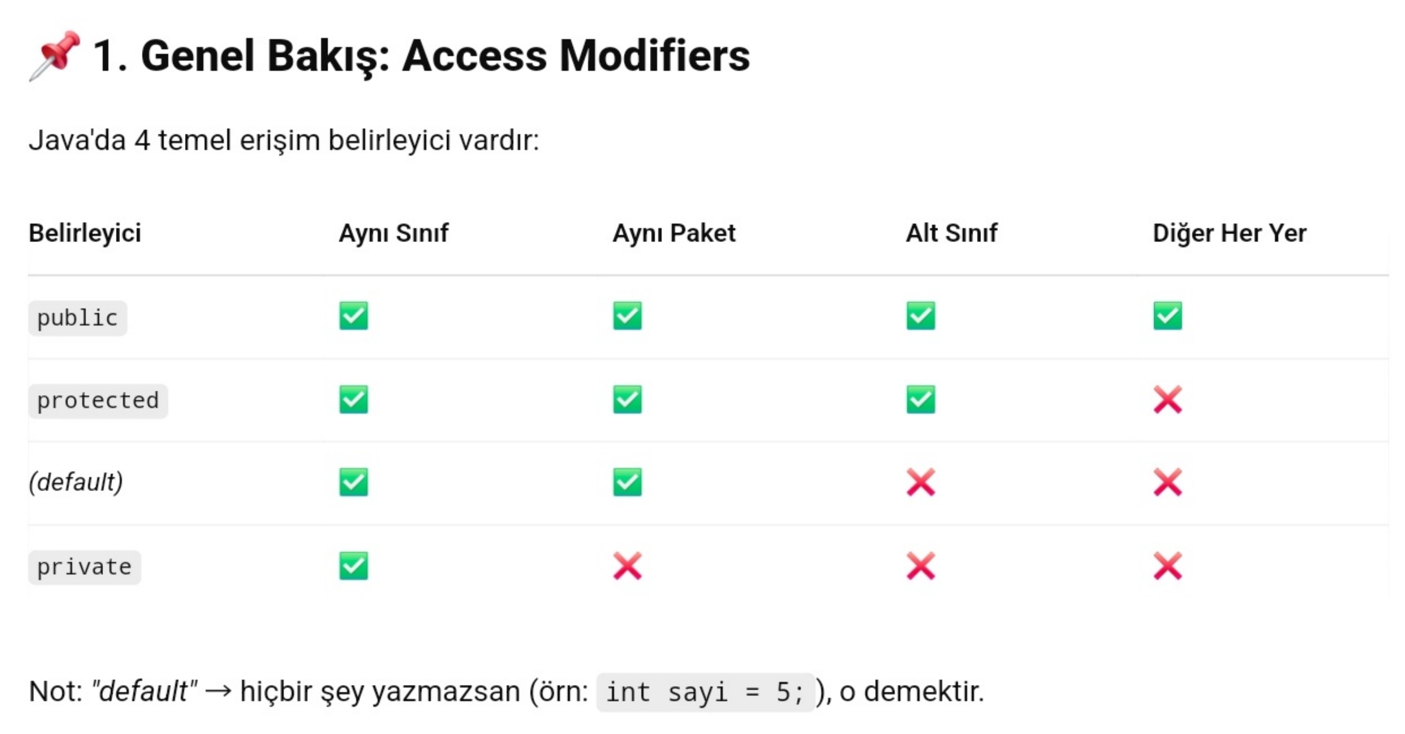
```java
public int seviye(int yeniDeger) {
    if (yeniDeger < 0)
        seviye = 1;
    else
        seviye = yeniDeger;
    return seviye;
}
```

Peki sadece **okumak** istersek ne yapacağız? Mesela:

```java
int x = a.seviye(); // ❌ çalışmaz çünkü parametre bekliyor
```

Bu durumda ayrı ayrı getter ve setter olmasının avantajı ortaya çıkar.

- private
+ public
# protected
~ package/default



📌 1. Genel Bakış: Access Modifiers

Java'da 4 temel erişim belirleyici vardır:

Belirleyici	Aynı Sınıf	Aynı Paket	Alt Sınıf	Diğer Her Yer
`public`	✅	✅	✅	✅
`protected`	✅	✅	✅	❌
(default)	✅	✅	❌	❌
`private`	✅	❌	❌	❌

Not: *"default"* → hiçbir şey yazmazsan (örn: `int sayi = 5;`), o demektir.

Paketler

v src
  v deneme8 (package)
    > A.Java (class)
  v deneme9
    > sınıfım.Java (class)

import package deneme8

denediğim sınıfın class'ı içindeyken. Yoksa A'daki sınıfları göremez.

private class tasarladığında ondan nesne yaratamazsın.

~~Math. matematik = new. Math();~~

double y = Math.max(2,5); ✓

sadece static
metotları
böyle noktayla
çağırabiliyorsun

isDeger

bir gelenek olarak, boolean olduğundan isDeger yazılır.

public boolean ~~get~~Deger(){
    return ?;
}

Dizi nesne yaratmak için:

A[] dizi = new A[10]; → şu an sadece 10 çocuğun adı var ama henüz doğmadılar.

for (int i=0; i<dizi.length; i++)

→ değişken gibi olduğundan parantez yok, string.length()'de var ama

dizi i = new A();

şimdi doğdular.

iki dairenin çaplarını yer değiştirme: (306. sayfa)

```
public static void swap2(Circle x, Circle y){
    double temp = x.radius
```

Immutable Class

İçinde yarattığın değişkenler sonradan değiştiremiyorsun.

1) Bütün değişkenler ve referans değerleri private olmalıdır.

2) Asla setter method olmamalıdır.

3- Hiçbir getter metod değeri sonradan değiştirilebilecek bir nesnenin referansını döndürmemelidir.

✗ Tüm değişkenler final ("const"=sabit) olmalıdır.

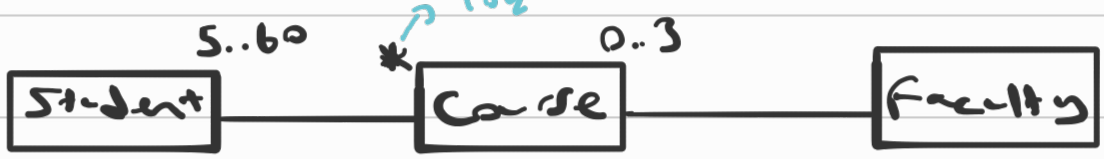
* "String" dersi de bi immutable class'tır.

★ Bir constructor başka bir constructor'ı çağıracaksa hemen ilk satırda "this" kullanmalıdır.

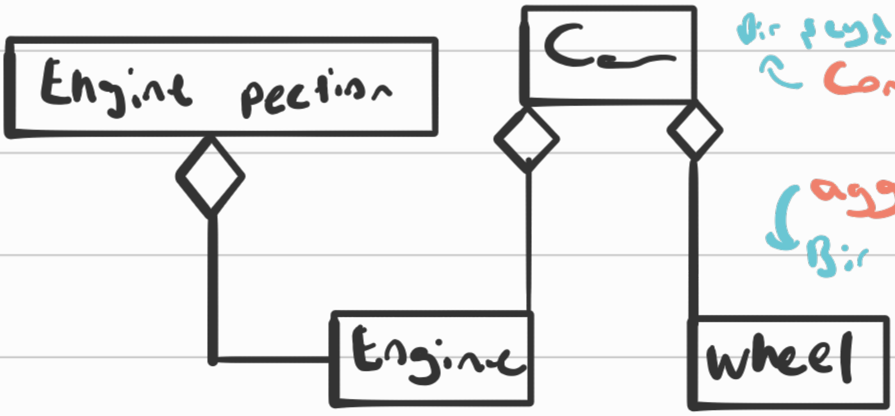
```
public class Kitap{
private String isim;
private String yazar;
protected int sayfaSayisi;

public String getIsim(){
return isim;
}

public void setIsim(String isim){
this.isim=isim;
}

public String getYazar(){
```

```
return yazar;
}

public void setYazar(String yazar){
this.yazar=yazar;
}

public void bilgiGoster(){
//burayı yazmaya üşendim geç sen fjcdjfjfrjct
}

}
```

Relation Diagram


5..60
*
0..3
Student
Course
Faculty

çok fazla sayıda

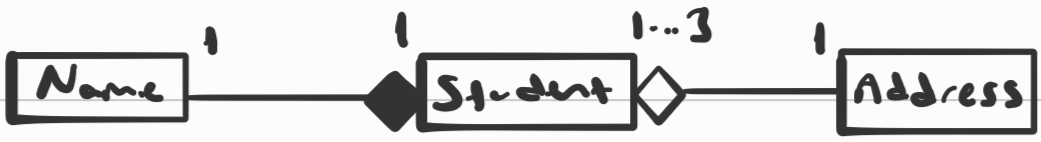

Engine pection
Car
Engine
wheel

Bir şeyden doğan

Composition (◆) varoluşsal bağımlılık var.

aggregation (◇) " " yok.

Bir araya gelme


1
1
1..3
1
Name
Student
Address

String karşılaştırma

```
String x="ali";

String y="dly";

System.out.println(x.compareTo.(y));
```

STACK (yığın)

LIFO(Last in first out, son giren ilk çıkar

pop(çıkarma)

push(yeni)

peek(en üst)

elements: int[]

size: int

StackOfMyIntegers()

StackOfMyIntegers(capacity: int)

empty(): boolean

peek(): int

```
public class StackOfMyInteger{
    private int[] elements;
    private int size;
               nesneye ait demek
    public static final int DEFAULT_CAPACITY


public StackOfMyInteger(int capacity){
    elements = new
```

```
public int peek(){
    return elements[size-1];
}
public void push(int value){
    if(size>=elements.length){
        int[] newElements = new int[elements.length*2];
        System.arraycopy(elements,0,newElements,0,
                                  elements.length);
        elements = newElements;
    }
    elements[size++]=value;
```

Slaytlardan

High-Level Language: Kodlarda bizim yazdığımız dil

Source Program'ı Machine Language'a çeviren Compiler.

Operating System: Windows gibi işletim sistemleri

Javada kodlar "Bytecode" diye bir şeye dönüştürülüyo. Bunun sayesinde her cihazda açılabiliyo.

Modifier → public, static, private gibi kelimeler

Statement → System.out.println(" "); → gibi ifadeler

Parse → bir yazıyı çözümlemek demek

int a;

String temp=JOptionPane.showInputDialog(null,"bir sayı gir");

a=Integer.parseInt(temp);

System.out.print(a+2); → öteki türlü a=42 desen mesela ekrana 422 yazardı string olduğundan dolayı. Biz bunu int'e çevirdik.

final double PI=3.14159; constants'lar için. Yani sabit, değişmeyenler

* 5.0/2 yazman lazım 5/2 yazarsan 2 verir int olduğu için

Miras Bırakma

class Hayvan{ → Ata sınıf (superclass)
:
void sesVer(){
}
}
class Kedi extends Hayvan{ → Yavru sınıf, alt sınıf (subclass)
↓
Hayvan'daki şeyler var. Buna ekleyecek.
.
.
@Override → (opsiyonel)
void sesVer(){ → aynı imzalı, overriding: Ata sınıfındakini reddedip başka bişey yapıyor
* super.sesVer(); → atadaki sesVer'i çağırır.
System.out.println("Miyav"); çıktısı şöyle olur → Hayvan ses verdi Miyav
}
}

dikkat atadaki constructor'u çağırmak istersen super(color, filled); gibi şeyler diyebilirsin.

```
Canli() {
  :
  Super();
}
```

Super(); → dersem, atasının constructorunu çağırır.

Polimorfizma

daima sağdakinin atası olmalıdır.

```
Top ft = new Futbol_topu();
```

→ polimorfik nesne

→ true yada false döndürür

örneği

```
if (x instanceof Futbol_topu)
    System.out.println("Gelen x nesnesi aslında bir futbol topu.");
else if (x instanceof Basket_topu)
    System.out.println("Gelen x nesnesi aslında bir basket topu.");
else
    System.out.println("Gelen x nesnesi aslında bir top");
```

Dynamic Binding

deklare edilen sınıf

gerçek sınıf

```
ChocolateCake c4 = new BirthdayCake();
```

~~c4.numSayisi = 8;~~ diyemem. çünkü, doğum günü pastasını sadece bi çikolatalı kek olarak görüyorsun

Bunu Düzeltmek için:

```
((BirthdayCake) c4).numSayisi = 8;
```

✓

explicit casting

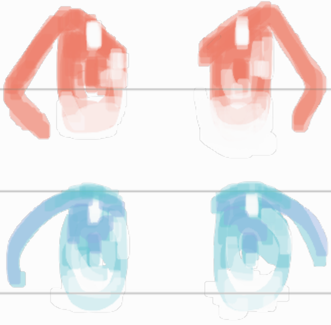
Kısaca Dynamic Binding, çalışan bir metodun hangi sınıfa ait olduğunu çalışma zamanında kontrol eder. Override edilmiş metotlar için kullanılır. Derleme zamanında bu keki düz kek gibi görüyorsun, ama kod çalışırken "ama bu doğum günü kekiymiş" diyip doğru sınıf kekindeki metodu çağırıyor.

```
Cake c  = new ChocolateCake();
Cake c1 = new Cake();
Cake c2 = new ChocolateCake();
Cake c3 = new BirthdayCake();
ChocolateCake c4 = new BirthdayCake();
```

derleme zamanında buna bakarım ilk, ki
c alan bir metodu var mı. (Evet var.)

c1.taste(cc); 1
c1.taste(c); 1
c2.taste(cc); 2
c2.taste(c); 2
c3.taste(cc); 4
c3.taste(c); 4
c4.taste(cc); 5
c4.taste(c); 4
parametre c

Şimdiii.... Kod derlerken önce deklare edilen class'a bakıp onun içinde "taste" metod arıyor. Eğer 1 taneyse zaten okey. Sonra çalışırken de gerçek sınıfta o metodun AYNISINI arıyor. O yüzden parametreye çok bakmıyor. c mi cc mi diye. Ama eğer deklare edilen classta iki tane taste varsa işte o zaman parametreye göre çağırıyor.



---

protected class → Defaulttaki her şey ama ekstradan farklı paketteki yavru sınıflardan erişilebilir bunlara.

```
try {
    System.out.println(a/b);
}
```

→ sıfıra bölmek istisnai mesela, hata verecek.

```
catch (ArithmeticException hata){
    System.out.println("Bölme hatası düştü");
}
```

System.out.println(hata.getMessage()); da diyebilirdin. Bu hatayı Java kendi yazar.

```
catch (ArrayIndexOutOfBoundsException hata) {
    System.out.println("Dizi sınırlarının
                       dışına çıkıldı!!!");
}
```

birden fazla catch verilebilir.

```
catch (InputMismatchException ime) {
    System.out.println("Girdi türü hatası!!!");
}
```

→ import etmek gerekir bunun için ama

RuntimeException da denebilirdi.. çünkü bunların hepsinin dedesi.. Ama exception daha güvenli

```
catch (Exception e) {
    System.out.println("Ata exception yakaladı!");
}
```

→ Bunu en üste yazmıyorsun!! Altındakilere erişemez. "Topu bir kez yakalayabilir"

.... (e.getLine()); deseydin hata veren satırı yazardı.

Ya da

```
catch (Exception e){
    if (e instanceof ArithmeticException)
        System.out.println("Aritmetik hata düştü");

    else if (e instanceof ArrayIndexOutOfBoundsException)
        ........

    else
        System.out.println("Bildiğim hata");
}
```

```
finally { → "Ne olursa olsun bu çalışacak!!"
    System.out.println("Meraba");
}
```

Random Sayı Alma

★ (0 <= Math.random() < 1.0)

(int)(Math.random()*10) → 0-9 arası bir sayı verir.

50+ (int)(Math.random()*50) → 50-99 arası ...

Kısaca, a+Math.random()*b → a ve a+b arası sayı üretir.
a+b dışında. a≤x<a+b

Pass By Value

```
public static void m(int number, int[] numbers){
    numbers[0] = number; → metodlarda diziler değiştirildikten
                           sonra dizi öyle kalır.
}
```

Array List

```
ArrayList<String> list = new ArrayList<>();
list.add("Apple");
list.add("Banana");
list.add(1,"Blueberry"); // diğerlerini kaydırıp bunu 1'e ekliceh. 1'i silip
                            yerine yazmıyor.
list.set(2,"Blueberry"); // bu yerine yazıyor
System.out.println("Element at index 1:" + list.get(
```

String Builder

StringBuilder sb = new StringBuilder("Hello ");

sb.append("Java"); → "Hello Java"

ekliyor

sb.insert(4, "Sam"); 4 konumdan yerine "Sam" koyar

"HellSamo Java"

sb.delete(1,5); → 1-5 arası karakterleri siliyor (1 ve 5 de dahil)

sb.deleteCharAt(5);

sb.replace(1,4,"ello"); → 1-4 arasına ello yazdı değiştirmek

sb.setCharAt(4,'a'); → 4'ü a yaptı

sb.reverse;

Override: Alt sınıf, atasından aldığı metodu kendine göre yeniden yazıyor.

- İsim ve Parametreler aynıdır. Sadece içi farklı

Overload: Kalıtımla çok bir alakası yok. Aynı sınıfta birden fazla aynı isimli metot olursa ama parametreleri farklıysa deniyor.

örn

```
public int topla(int a, int b){
} ~~~ ...
public int topla(double a, double b){
} ~~~ ...
```

- Class'lara da "final" diyebilirsin. Dersen çocuğu olamaz.

```
final class Math { gibi!
    . - - -
}
```

- Paket dışından kalıtım ile erişmek istiyorsan Protected class yapabilirsin. Public de olur. Ötekiler olmaz, default ve Private

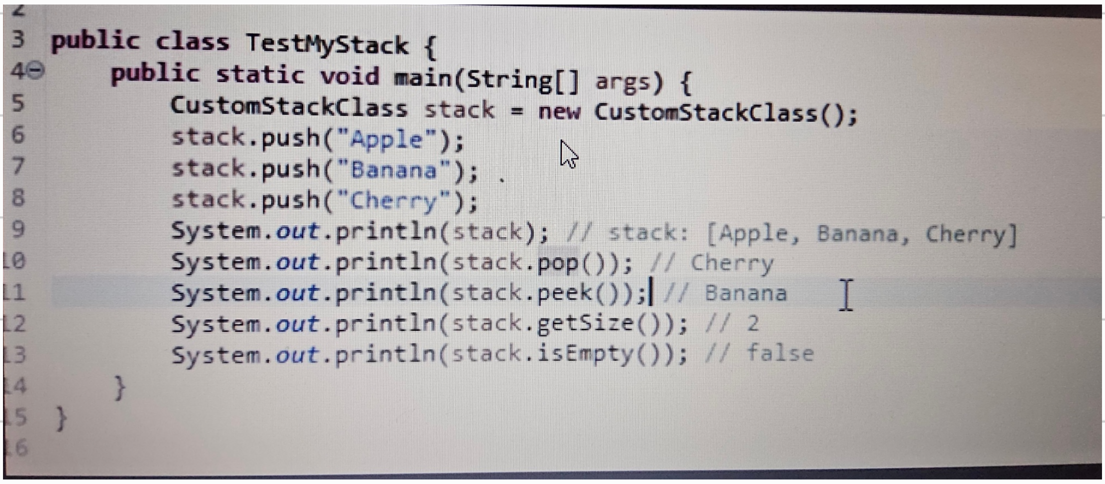
- Bir değişken, metodun içinde oluşturulursa, metot bittikten sonra o değişkene bir daha ulaşılamaz. O yüzden, değişken metoda parametre olarak iletilmelidir.

Stack (yığın)

Son giren ilk çıkar.

- push(item) → yığına eleman ekliyor
- pop() → en üstteki elemanı çıkarıyor
- peek() → en üstteki elemana bakıyor ama çıkarmıyor
- isEmpty() → yığın boş mu?
- size() → kaç eleman var?

kod



```
public class TestMyStack {
    public static void main(String[] args) {
        CustomStackClass stack = new CustomStackClass();
        stack.push("Apple");
        stack.push("Banana");
        stack.push("Cherry");
        System.out.println(stack); // stack: [Apple, Banana, Cherry]
        System.out.println(stack.pop()); // Cherry
        System.out.println(stack.peek()); // Banana
        System.out.println(stack.getSize()); // 2
        System.out.println(stack.isEmpty()); // false
    }
}
```

File Class

absolute file name = directory path + relative file name

"C:\\Deneme\\meth.txt" → "C:\\Deneme" + "meth.txt"

Javada çift ters slash kullanılır. Tek ters slash'ın başka bir anlamı vardır çünkü.

+File (pathname: String)
+File(parent:String, child:String)

Javada tarihler için "long" değişkenler kullanılır.

```
import java.io.File;
public class . . . . . {
public static void main(String[] args){
    File deneme=new File("C:\\Deneme\\");
    File deneme=new File("C:\\Deneme\\","meth.txt");
    File dosya =new File(deneme,"meth.txt");

    System.out.println("C:\\—  klasörü var mı? " + deneme.exists());
```

deneme.exists() → true/false döndürür

```
String[] dosyalar = deneme.list();
for(String ali: dosyalar){
    System.out.println(ali);
}
```

tek tek klasördeki dosyaları yazdırır.

dosya.canWrite() → bir metin belgesi yazılabilir mi? Salt okunur mu ona bakar.

dosya.length()

dosya.isDirectory() → dosya mı ona bakar (t-f)

dosya.delete()

```
dosya.renameTo(new File(deneme,"yeniad.txt"));
```

→ adını değiştirir

```
File yeni_klasör =new File(deneme,"YENIBIRKLASOR");
yeni_klasör.mkdirs();
```

mkdirs → birden çok yaratacaksan

PrintWriter → eski dosyaları silmesi açısından tehlikelidir. Eski dosyanın içindekileri silip yerine yeni bilgiyi yazar.

```
try{
 PrintWrite pw = new PrintWriter(metin);
   // FileNotFound hatası bir checked exceptiondır.
 for(int i=0;i<10;i++)
    pw.println("MERABA");
 pw.close();
}
catch(Exception e){ }
```

→ Ya da bunu yazmak yerine şöyle yapabilirsin: try with resources

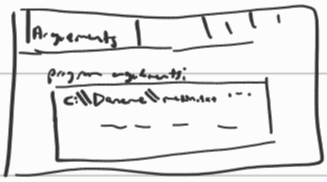
```
try(
    PrintWrite pw = new PrintWriter(metin)){
```

```
Scanner okuyucu=new Scanner(dosyam);
okuyucu.useDelimiter("#");
while(okuyucu.hasNext())
```

cmd yerinden komut vermek için, Eclipse'de run-yap

Run ∨

Run configurations ∨



Wow

Abstract Methods

```
abstract class GeometricObject{ //soyut sınıf
  public abstract double alanHesapla(); //soyut metot
}
```

- Soyut metotlar sadece soyut sınıflarda veya arayüzlerde bulunabilir.
- Soyut sınıflarda değişkenler, sabitler bulunabilir
- Soyut sınıf bir taslak sınıftır.
- Bir soyut sınıftan miras alan sınıf(lar) soyut sınıftaki soyut metotları gerçekleştirmelidir/ezmelidir. Eğer bu olmazsa, yavru da soyut olur.

ABC

- Soyut sınıflardan nesne yaratılamaz.
- Ata sınıftaki somut metod, yavru sınıftaki soyut metotla ezilebilir.

Soyut sınıf/metotların isimleri UML diyagramında italik yazılır.

- Soyut metotlar static olamaz.

«interface»

```
Class Chicken extends Animal implements Edible {
```

JavaFX

```
class Pencerem extends JFrame{
    Pencerem(){

        super("Benim programım"); → başlık
        int en = 600, boy = 750; piksel
yazmasan da olur bu arada ← this.setSize(en,boy);
        this.setResizeable(false); → insanlar pencerenin boyutuyla oynayamasın
        this.setDefaultCloseOperation(EXIT_ON_CLOSE); → pencere kapanınca program sonlansın.
        this.setVisible(true); → pencere görünür mü olsun?
                          En son yazılmalı!!
    }

}
main( ){
new Pencerem(); → direkt böyle diyebilirsin. Pencerem x = new.....
                  dersen de onu kullanmayacağın bir x nesnesi
                  yaratmış olursun.
```

Layout Manager

```
dosyala = new.JButton("Dosyala");

dosyala.setBounds(380,50,150,25);
                          ↓
                       genişlik

dosyala.addActionListener

class Klak implements
```

Lambda Expression → Java 8'le birlikte geldi. Interface'imiz sadece ama sadece 1 tane soyut metot içeriyorsa yazılabilir.

JCheckBox[] jcb; → ☑

jcb = new JCheckBox[3];

Radyo butonlar ●

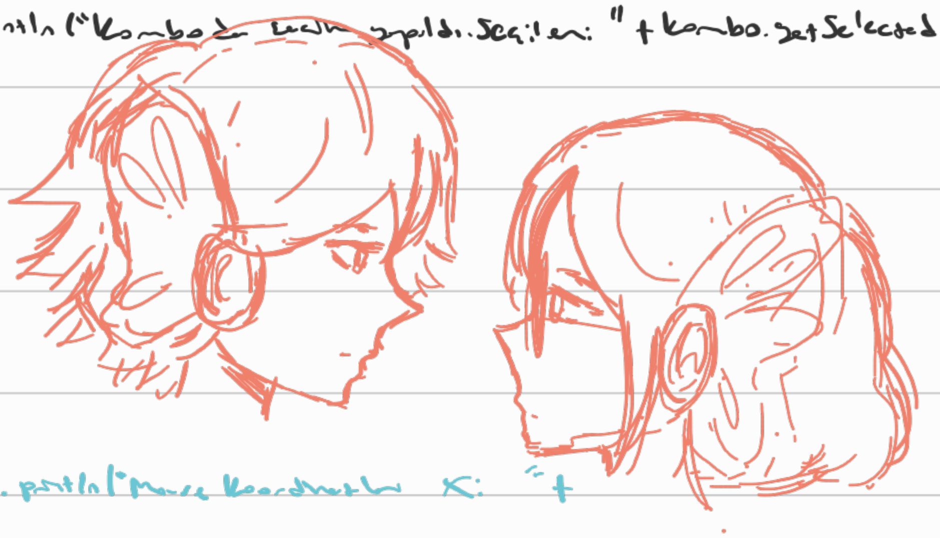
```
jrb = new JRadioButton[4];
```

Kombo Box

```
kombo = new JComboBox<String>(new String[] {"Galatasaray", "Fenerbahçe", "Beşiktaş"});
```

generic ifade (yazılan kısmı değil)

```
kombo.addItemListener(
    e -> {System.out.println("Komboda seçim yapıldı. Seçilen: " + kombo.getSelected
```



Mouse Listener

mouseClicked
mousePressed
mouseReleased

Mouse Motion Listener

mouseDragged
mouseMoved → System.out.println("Mouse Koordinatları X: " +

Key Listener

keyTyped klavye tuşlar
keyPressed
keyReleased

JPanel
JButton
JLabel
JTextField
} Sınavda önemli olanlar ve ezberlenmesi gerekenler

Nested Class → iç içe geçmiş sınıf

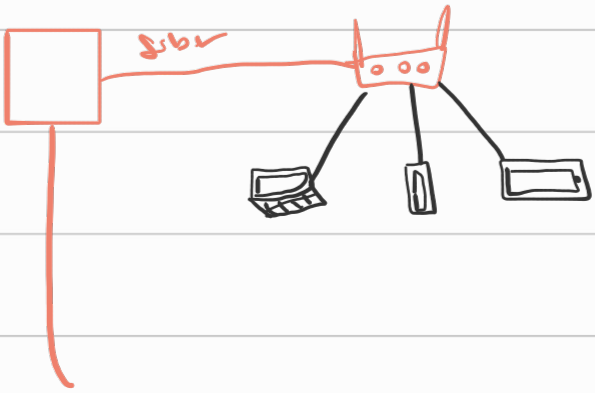
1 → New
2 → Runnable start() çağrıldı
3 - Running çalışıyor.
4 - Blocked/waiting bekliyor
5 - Terminated/dead son durumda

```
public class Thread implements Runnable

        t1.run();
        t1.start();

    Thread.sleep()
```

Networks

ISP (Internet Service provider) (internet sağlayıcısı, Türk Telekom gibi)

LAN (Local Area Network)



DNS

btu.edu.tr	x.y.t.w

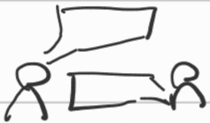
cmd

```
c:\Users\~~~~> ping www.btu.edu.tr

    pinging www.btu.edu.tr [79.123.219.23] with 32 bytes of data:
```

bunu google'a direkt yazarsan btü sitesine götürür

```
public class Server {

    public static void main(String[] args) throws IOException{

        int dinleme_portu = 8465;

        ServerSocket ss = new ServerSocket(dinleme_portu);

        Socket s =
```

"Man in the middle" →



→ adam mesajları group değiştirir

★ Bir sınıf hem implements hem extends kullanabilir.

★ Sadece 1 sınıf extend edilebilir ama birden fazla interface implement edebilirsin.

class C extends A implements B, C, D { } gibi

★ Bir Java dosyası içinde sadece 1 tane public class olabilir. yoksa hata veriyo maalesef. Ya public yapmıcan ya da ayrı ayrı Java dosyaları oluşturucan.

class kedi
class hayvan
public class main

gibi, ya da

Hayvan.java → public class hayvan
Kedi.java → public class kedi
⋮

★ Thread sınıfı → extend edilir
Runnable interface'i → implement edilir.

Thread

```
t1.start();
t2.start();
```

} run() metodunu başlatırlar. run() doğrudan ÇAĞRILMAMALIDIR. Yoksa yeni bir thread oluşturulmaz.

★ Thread sınıfı da Runnable arayüzünü implemente eder. Ama kendisi bir arayüz değildir.

sleep metodu

```
try {
    Thread.sleep(500);
}
catch(InterruptedException e){
```

→ milisaniye cinsinden

```
    System.out.println(name + " kesmeye uğradı!");
}
```

Eğer sınıfın başka şeyleri extend etcekse aynı anda Thread sınıfını extend edemez. Bu durumda Runnable arayüzünü implement edebiliriz.

Hemen hemen aynı şeyi yaparsa:

```
class sınıfım implements Runnable {
    public void run() {
        .......
    }
}
```

Runnable ile:

```
Thread t1 = new Thread(new MyTask("Görev 1"));
```

Thread ile:

```
MyThread t1 = new MyThread("Görev 1");
```

extends Thread → zaten bir thread olduğu için doğrudan new MyThread(...)

implements Runnable → Bir görev tanımı yaptın sadece. Onu bir Thread ile çalıştırmak gerekir.

new Thread(new MyTask(...))

join metodu

Bir thread'in bitmesini bekler. Sıralı çalışmasını sağlar.

```
t1.start();
t1.join(); → t1 bitene kadar bekle
t2.start();
t2.join(); → t2 bitene kadar bekle
```

`Thread.currentThread().getName()` → Thread'in ismini alıyo

Klonlama

```
class Oda {....—

class Ev implements Cloneable {
    int i;  ←——————— bunun gibi
    Oda oda; → Oda oluşturacağını bildirmişsin gibi düşün.
    Ev(int i) {
        this.i = i;
        oda = new Oda(20);
    }
              SIĞ KOPYALAMA (shallow copy)
    public Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}

Main {
    Ev ev1 = new Ev(10);
    Ev ev2 = (Ev) ev1.clone();
```

```
System.out.println(ev1==ev2);
```

// false ise iki farklı 2 nesne demek. pointerları farklıdır.

```
System.out.println(ev1.oda.m2);
System.out.println(ev2.oda.m2);
```

I odanın metrekare

ikisi de 20 döndürür

AMA

`ev1.oda.m2 = 30;` yapıp bir daha yazdırırsam

```
System.out.println(ev1.oda.m2);
```
→ 30

```
System.out.println(ev2.oda.m2);
```
→ 30

der. ev2 de değişti yani

AMA

`ev1.i = 80;` dersem

```
System.out.println(ev1.i);
```
→ 80

```
System.out.println(ev2.i);
```
→ 10 hala 10 kalır.

Sığ kopyalamada nesneye bağlı olan nesneler kopyalanmaz.

Derin kopyalamada diğer nesneler de kopyalanır. (deep copy)

```
public Object clone() throws CloneNotSupportedException{
    Ev kopya = (Ev) super.clone();
    kopya.oda =(oda) this.oda.clone();
    return kopya;
}
```

sığ kopyasını

→ Evin sığ kopyalamasını yaptık.

→ odayı da klonladık.

→ oda olduğunu belirttik

VE

```
class Oda implements Cloneable{
    ⋮
    public Object clone() throws CloneNotSupportedException{
        return super.clone();
    }
}
```

da demem gerekir.

VE içine de klonlama metodu yazmalıyım

Eğer Oda'nın içinde de bir nesne yoksa direkt sığ kopyalama yeterli olacaktır; varsa onu da derin kopyalama yapmak

```
JPanel[] paneller;
```

[illegible] [illegible] gibi

```
this.setLayout(new GridLayout(3,1));
```

3 satır
1 sütun

```
Color[] renkler = {Color.RED, Color.
                          GREEN, Color.BLUE}
```

```
import javax.swing.JFrame;
import javax.swing.JPanel;
import java.awt.Color;
```

```
paneller = new JPanel[3];

for(int i = 0; i<paneller.length; i++)
{      paneller[i] = new JPanel();
       paneller[i].setBackground(renkler[i]);
      this.add(paneller[i]);
}
```

```
import java.io.File;
```